

HESTIA

Arquitetura inicial do HestIA

Comparação entre monólito em camadas e monólito modular

SCRUM-102 | ADR-001
Projeto Integrado IV
ESPM

Documento de modelagem e arquitetura

- Status: Aceita
- Data: 2026-08-30
- Tarefa: SCRUM-102
- Decisão: adotar um monólito modular, com camadas internas em cada módulo de negócio

Contexto

O HestIA é uma plataforma corporativa para governança, monitoramento e uso responsável de Inteligência Artificial. O MVP está sendo desenvolvido por uma equipe pequena, em Java 21, Spring Boot, Spring Web MVC, Spring Data JPA e PostgreSQL.

O repositório atual já está organizado principalmente por áreas de negócio:

```
br.com.hestia
+--- empresa
+--- departamento
+--- usuario
+--- politica
+--- ferramenta
```

Dentro dos módulos existentes são usadas as camadas `controller`, `service`, `repository`, `dto` e `model`. A decisão arquitetural deve preservar a simplicidade de implantação do MVP sem permitir que o código se transforme em um único conjunto de classes fortemente acopladas.

Forças que influenciam a decisão

- Equipe pequena e ainda consolidando práticas de desenvolvimento.
- Prazo acadêmico e necessidade de entregar um MVP executável.
- Domínios que já possuem limites reconhecíveis.
- Necessidade de manter regras de empresa, departamento, usuário, política e ferramenta isoladas.
- Baixa necessidade atual de escalabilidade independente por domínio.
- Necessidade de testes e refatorações com baixo custo operacional.

Alternativas consideradas

Alternativa A - Monólito em camadas globais

Estrutura típica:

```
br.com.hestia
+--- controller
+--- service
+--- repository
+--- dto
+--- model
```

Vantagens

- Estrutura simples e conhecida.
- Curva de aprendizado inicial pequena.
- Pouca configuração adicional.

Desvantagens

- Classes de domínios diferentes ficam misturadas nas mesmas pastas.
- O acoplamento entre funcionalidades cresce com facilidade.
- A responsabilidade de cada domínio se torna menos evidente.
- Alterações em uma funcionalidade tendem a atravessar várias pastas globais.
- Dificulta uma possível extração futura de módulos.

Alternativa B - Monólito modular

Estrutura proposta:

```
br.com.hestia
+--- empresa
|   +--- controller
|   +--- dto
|   +--- model
|   +--- repository
|   +--- service
+--- departamento
+--- usuario
+--- politica
+--- ferramenta
```

Vantagens

- Mantém juntas as classes que pertencem à mesma capacidade de negócio.
- Torna os limites e responsáveis pelos módulos mais claros.
- Reduz referências acidentais entre áreas sem relação.
- Permite continuar com uma única aplicação e um único processo de implantação.
- Facilita testes por módulo e uma eventual evolução arquitetural.
- É compatível com a organização que o repositório já começou a adotar.

Desvantagens

- Exige disciplina para respeitar os limites entre módulos.
- Pode haver duplicação de contratos ou mapeamentos se as responsabilidades não forem claras.
- Ainda não oferece escalabilidade ou implantação independente por módulo.
- Dependências circulares podem surgir sem regras de arquitetura.

Comparação

Critério	Monólito em camadas globais	Monólito modular
Simplicidade inicial	Alta	Alta a média
Clareza dos domínios	Baixa	Alta
Isolamento de regras	Baixo	Médio a alto
Adequação ao repositório atual	Baixa	Alta
Custo operacional	Baixo	Baixo
Evolução do MVP	Limitada	Favorecida
Risco de acoplamento crescente	Alto	Médio
Extração futura de módulos	Difícil	Mais simples

Decisão

O HestIA adotará monólito modular com camadas internas por módulo de negócio.

A aplicação continuará sendo construída, testada e implantada como uma única unidade. Cada capacidade de negócio terá seu próprio pacote raiz e poderá conter suas próprias camadas. Não serão criados microserviços no MVP.

Regras arquiteturais

1. Cada funcionalidade deve pertencer a um módulo de negócio explícito.
2. Controllers acessam services; controllers não acessam repositories diretamente.
3. Services concentram casos de uso e regras de negócio.
4. Repositories pertencem ao módulo que é responsável pela entidade.
5. Entidades JPA não devem ser duplicadas em módulos diferentes.
6. DTOs definem os contratos de entrada e saída da API; entidades não devem ser expostas diretamente quando contiverem dados sensíveis.
7. Um módulo só deve acessar outro por uma interface ou serviço claramente definido quando houver regra de negócio envolvida.
8. Dependências circulares entre módulos não são permitidas.
9. Recursos compartilhados devem ser pequenos e tecnicamente neutros; não devem formar um módulo genérico com regras de negócio misturadas.
10. Decisões que alterem esses limites devem ser registradas em uma nova ADR.

Dependências iniciais entre módulos

```
departamento --> empresa
usuario -----> empresa
usuario -----> departamento
politica -----> empresa
ferramenta ----> empresa
```

As setas representam dependências de domínio já existentes ou previstas para o MVP. O escopo dos dados deve sempre respeitar a empresa do usuário autenticado.

Consequências

Consequências positivas

- O desenho atual do código passa a ter uma regra formal.
- Novas funcionalidades ganham um local previsível.
- A equipe pode trabalhar em módulos diferentes com menos conflitos.
- O custo de infraestrutura permanece adequado ao MVP.

Consequências negativas e riscos

- A separação depende de revisão e testes arquiteturais futuros.
- O banco de dados continua compartilhado por todos os módulos.
- Uma alteração interna ainda pode afetar a aplicação inteira.
- A equipe precisará corrigir inconsistências já introduzidas no repositório.

Dívida arquitetural identificada

Existem atualmente duas classes JPA chamadas `PoliticaUso`, nos pacotes `br.com.hestia.politica` e `br.com.hestia.politicauso`, ambas mapeadas para a tabela `politicas_uso` e com modelos diferentes.

Essa duplicação viola o limite definido nesta ADR e pode causar conflito de mapeamento, comportamento inconsistente e perda de dados. O módulo canônico deverá ser `br.com.hestia.politica`; a versão duplicada deverá ser analisada e removida em uma tarefa específica, preservando os campos aprovados pelo domínio.

Validação da decisão

A decisão será considerada aplicada quando:

- cada entidade possuir um único módulo responsável;
- novos recursos seguirem a estrutura por domínio;
- controllers não acessarem repositories diretamente;
- não houver dependências circulares entre módulos;
- a equipe registrar exceções ou mudanças relevantes em novas ADRs.

Quando revisar esta ADR

Reavaliar a decisão se ocorrer pelo menos uma destas condições:

- necessidade comprovada de implantar ou escalar um módulo separadamente;
- crescimento da equipe que exija autonomia operacional por domínio;
- requisitos de segurança ou disponibilidade incompatíveis com uma única aplicação;
- aumento relevante do tempo de build ou implantação;
- dependências entre módulos que impeçam a evolução independente do produto.